



WEEK 2 - PYTHON PROGRAMMING

Operators, Strings & Conditionals

Expressions, text processing and decision making

Theory 2 h - Practice 2 h

Instructor: MinGi Kyung

Today's Agenda

- Part 1: Theory (2 h)
- Arithmetic, comparison and logical operators
- String indexing, slicing and methods
- Decision making with if, elif and else

Part 2: Practice (2 h)

Lab 1: String manipulation

Lab 2: Grade classifier

Lab 3: Rock-paper-scissors

Lab 4: Leap-year calculator

Four short Python programs



PART 1 - THEORY

Operators, Strings & Conditionals

Calculate with operators and compare values

Extract and transform text

Choose a branch based on a condition

Expression, Value & Type

- An expression is evaluated to produce a value.
- The type determines which operations are available.
- Predict both the value and its type before running.
- Use `type()` to check your prediction.

```
print(8 + 2)           # 10
print(type(8 + 2))     # int
print(8 / 2)           # 4.0
print(type(8 / 2))     # float
print("8" + "2")       # 82
print(type("8" + "2")) # str
```

Review: Input, Types & Output

- `input()` returns a string, even for numeric input.
- Convert numeric text with `int()` or `float()`.
- An expression combines values and operators.
- Predict the output before running the code.

```
a = "12"
b = "3"
print(a + b)           # 123
print(int(a) + int(b)) # 15

price = 12.5
quantity = 3
total = price * quantity
print(f"Total: {total:.2f}")
# Total: 37.50
```

Tracing Input Conversion

- Treat raw as text read from input().
- strip() removes surrounding whitespace.
- int() creates an integer from numeric text.
- Converting a value does not change raw.
- Explain why the two additions differ.

```
raw = " 12 "  
clean = raw.strip()  
number = int(clean)  
print(repr(raw))      # ' 12 '  
print(repr(clean))    # '12'  
print(number + 3)     # 15  
print(clean + "3")    # 123
```

Arithmetic Operators

- + and - add and subtract. * multiplies.
- / performs division. // floors the quotient.
- % gives the remainder. ** raises a power.
- Floor division rounds down: -7 // 3 is -3.
- Division or remainder by zero raises an error.

```
a, b = 7, 3
print(a + b)    # 10
print(a - b)    # 4
print(a * b)    # 21
print(a / b)    # 2.333...
print(a // b)   # 2
print(a % b)    # 1
print(a ** b)   # 343

minutes = 135
print(minutes // 60) # 2 hours
print(minutes % 60)  # 15 minutes
```

Exercise: Hours & Student Groups

- Convert 185 minutes into hours and minutes.
- Pack 29 students into groups of 4.
- Find the number of full groups and students left.
- Use // and % in both calculations.
- Try your solution before viewing the next slide.

```
minutes = 185
students = 29
group_size = 4

# Compute hours and remaining minutes.
# Compute full groups and students left.

# Expected: 3 hours, 5 minutes
# Expected: 7 groups, 1 student left
```

Solution: Quotient & Remainder

- The quotient counts complete units.
- The remainder counts what is left over.
- Check that quotient * divisor + remainder equals the original value.
- Repeat with 180 minutes and 28 students.

```
minutes = 185
hours = minutes // 60
left = minutes % 60
print(hours, left)          # 3 5
print(hours * 60 + left)    # 185
```

```
students = 29
groups = students // 4
left = students % 4
print(groups, left)         # 7 1
print(groups * 4 + left)    # 29
```

Comparison Operators

- `==` tests equality. `!=` tests inequality.
- `<`, `<=`, `>` and `>=` compare values.
- Comparisons return `True` or `False`.
- `=` assigns a value. `==` compares values.
- A chained comparison can check a range.

```
score = 85
print(score == 85) # True
print(score != 85) # False
print(score >= 60) # True
print(score < 60)  # False
print(0 <= score <= 100) # True

print("85" == 85)  # False
# Convert input before comparing.
```

Inclusive & Exclusive Boundaries

- An inclusive boundary accepts the endpoint.
- An exclusive boundary rejects the endpoint.
- Compare the results at 0 and 100.
- For a score range, test just below, at and just above each endpoint.

```
score = 100
print(0 <= score <= 100) # True
print(0 < score < 100)   # False
```

```
score = 0
print(0 <= score <= 100) # True
print(0 < score < 100)   # False
```

```
score = 101
print(0 <= score <= 100) # False
```

Logical Operators

- and requires both conditions to be true.
- or requires at least one condition to be true.
- not reverses the truth value.
- Python short-circuits: it skips an operand when the result is already determined.

```
age = 20
has_id = True
print(age >= 18 and has_id) # True
print(age < 18 or has_id)   # True
print(not has_id)           # False

n = 0
print(n != 0 and 10 / n > 2)
# False: 10 / n is never evaluated.
```

Boolean Results & Short-Circuiting

- and is true only when both inputs are true.
- or is false only when both inputs are false.
- In A and B, Python skips B when A is false.
- A guard can make a later expression safe.

```
print(True and True)    # True
print(True and False)   # False
print(False and True)   # False
print(False and False)  # False
print(False or True)    # True
print(False or False)   # False

text = ""
print(text != "" and text[0] == "A")
# False: text[0] is not evaluated.
```

Precedence & Assignment Updates

- `**` binds more tightly than `*` `/` `//` `%`, which bind more tightly than `+` `-`.
- Comparisons come before `not`, then `and`, then `or`.
- Parentheses make the intended grouping clear.
- `+=`, `-=` and `*=` update an existing variable.

```
print(2 + 3 * 4)      # 14
print((2 + 3) * 4)    # 20
print(-2 ** 2)        # -4
print((-2) ** 2)      # 4
```

```
score = 80
score += 5
print(score)          # 85
```

```
# Which output do you expect?
print(True or False and False)
# True
```

Exercise: Parentheses & Updates

- Predict each result before revealing the comments.
- Multiplication happens before addition.
- Update statements run in written order.
- Explain how parentheses change the last result.

```
print(10 - 2 * 3)      # 4
print((10 - 2) * 3)    # 24

x = 5
x += 3
x *= 2
print(x)               # 16

print(True or False and False)
# True
print((True or False) and False)
# False
```

Strings & Indexing

- A string is a sequence of characters.
- The first character has index 0.
- Negative indices count from the end.
- `len(text)` returns the number of characters.
- An index outside the string raises `IndexError`.

```
text = "Python"
#      012345
print(len(text)) # 6
print(text[0])   # P
print(text[2])   # t
print(text[-1])  # n
print(text[-2])  # o

# text[6] raises IndexError
# ""[0] also raises IndexError
```

Valid Indices & Empty Strings

- For length n , nonnegative indices are 0 through $n - 1$.
- For a nonempty string, -1 selects the last character.
- A one-character string has the same first and last character.
- An empty string has no valid character index.

```
text = "Code"
print(len(text)) # 4
print(text[0])   # C
print(text[3])   # e
print(text[-1])  # e

text = "A"
print(text[0] == text[-1]) # True

# "Code"[4] raises IndexError.
# ""[0] raises IndexError.
```

String Slicing

- `text[start:stop]` includes start and excludes stop.
- An omitted start begins at index 0. An omitted stop goes to the end.
- A third value sets the step.
- Slices tolerate bounds beyond the string.
- Strings are immutable. Create a new string to change the text.

```
text = "Python"
print(text[0:2]) # Py
print(text[2:])  # thon
print(text[:4])  # Pyth
print(text[::2]) # Pto
print(text[::-1]) # nohtyP
print(text[:99]) # Python

# text[0] = "J" is invalid.
text = "J" + text[1:]
print(text)      # Jython
```

Exercise: Reading Slices

- Write the selected indices on paper first.
- The stop index is excluded.
- With a positive step, start equal to stop gives an empty string.
- `repr()` makes an empty result visible as quotes.

```
text = "PYTHON"
print(text[1:4])      # YTH
print(text[:3])       # PYT
print(text[-3:])      # HON
print(text[1::2])     # YHN
print(text[::-1])     # NOHTYP
print(repr(text[2:2])) # ''
print(text[:99])      # PYTHON
```

String Methods & Membership

- `strip()` removes whitespace at both ends.
- `lower()` and `upper()` change letter case.
- `replace(old, new)` returns replaced text.
- `split()` separates text into a list of strings.
- `in` checks whether a substring occurs. Methods return results without changing the original string.

```
raw = "  Hello Python  "
clean = raw.strip()
print(clean)           # Hello Python
print(clean.lower())   # hello python
print(clean.upper())   # HELLO PYTHON
print(clean.replace("Python", "AI"))
# Hello AI
print(clean.split())
# ['Hello', 'Python']
print("Python" in clean) # True
print("python" in clean) # False
```

A Text-Cleaning Pipeline

- Store intermediate results while learning.
- `strip()` affects only the ends of the string.
- `replace()` can change spaces inside the string.
- Each method returns a new string.
- The original raw value remains available.

```
raw = "  Hello Python  "  
clean = raw.strip()  
lowered = clean.lower()  
slug = lowered.replace(" ", "-")  
print(repr(raw))  
# '  Hello Python  '  
print(clean)      # Hello Python  
print(lowered)    # hello python  
print(slug)       # hello-python
```

Joining Text & Formatting Values

- + concatenates strings. * repeats a string.
- str() converts a number to text.
- An f-string embeds expressions inside braces.
- .2f displays two digits after the decimal point.
- Normalize user input before comparing text.

```
name = "Mina"
score = 87.5
print("Hi, " + name) # Hi, Mina
print("-" * 5)       # -----
print("Score: " + str(score))
print(f"{name}: {score:.2f}")
# Mina: 87.50

answer = "  YES  "
print(answer.strip().lower() == "yes")
# True
```

Worked Example: A Receipt

- Keep prices numeric while calculating.
- Format the result when displaying it.
- A format such as `.2f` changes the displayed text.
- It does not replace the numeric value stored in `total`.

```
item = "Notebook"
price = 2.5
quantity = 3
total = price * quantity

print(f"Item: {item}")
print(f"Quantity: {quantity}")
print(f"Total: {total:.2f}")
print(total) # 7.5, still numeric
```

Conditionals: if & else

- if runs an indented block when its condition is true.
- else runs when that condition is false.
- End the if and else headers with a colon.
- Use four spaces for each indentation level.
- Code after the block runs regardless of the branch.

```
age = 17
if age >= 18:
    print("Adult")
else:
    print("Under 18")
print("Check complete")
```

```
# Output:
# Under 18
# Check complete
```

Tracing a Branch

- Evaluate the condition before reading the branch.
- Exactly one of the if/else blocks runs.
- The unindented final print always runs.
- Change age to 18 and trace the output again.

```
age = 17
print("Start")
if age >= 18:
    print("Adult")
else:
    print("Under 18")
print("Done")
```

```
# Start
# Under 18
# Done
```

Multiple Branches with elif

- elif checks another condition if earlier conditions were false.
- The first matching branch runs.
- An optional else handles all remaining cases.
- Check grade thresholds from highest to lowest.
- Separate if statements can run multiple blocks.

```
score = 85
if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
elif score >= 60:
    grade = "D"
else:
    grade = "F"
print(grade) # B
```

Separate if Statements & elif

- Separate if statements each evaluate their own condition.
- An if/elif chain stops at the first matching branch.
- Use a chain for mutually exclusive grade labels.
- Use separate checks when multiple messages are intended.

```
score = 95
if score >= 90:
    print("A")
if score >= 80:
    print("B")
# Both A and B are printed above.

if score >= 90:
    print("A")
elif score >= 80:
    print("B")
# Only A is printed by this chain.
```

Validation & Nested Conditions

- Validate a value before using it.
- A nested if runs inside another branch.
- Use a guard to avoid dividing by zero.
- For these labs, assume numeric input has the requested format.
- We will handle conversion errors with exceptions later.

```
a = float(input("First number: "))
b = float(input("Second number: "))

if b == 0:
    print("Cannot divide by zero")
else:
    result = a / b
    if result < 0:
        print("Negative result")
    print(f"Result: {result:.2f}")
```

Validation Before Classification

- Reject values outside the allowed domain first.
- Keep the normal decision inside the valid case.
- Test a negative value, zero and a valid positive value.
- Numeric conversion errors are outside this exercise.

```
tickets = -1
if tickets < 0:
    print("Invalid quantity")
else:
    if tickets == 0:
        print("No booking")
    else:
        print(f"Booked: {tickets}")

# -1: Invalid quantity
# 0: No booking
# 2: Booked: 2
```

Common Mistakes

- Use `==` for equality, rather than assignment `=`.
- Missing colons or indentation break a block.
- Convert text before comparing numeric values.
- `"False"` is a nonempty string, so `bool("False")` is `True`.
- Compare both alternatives explicitly when using `or`.

```
choice = "paper"

# Incorrect: "scissors" is truthy.
if choice == "rock" or "scissors":
    print("This always runs")

# Correct:
if (choice == "rock"
    or choice == "scissors"):
    print("Rock or scissors")
```

Debugging Exercise: A Discount

- A discount applies only to student or senior customers.
- The current program discounts every customer.
- Explain which part of the condition is always truthy.
- Repair the condition and test three customer types.

```
customer = "adult"
price = 100
if customer == "student" or "senior":
    price *= 0.8
print(price) # 80.0: incorrect for adult

# Expected after repair:
# adult: 100
# student: 80.0
# senior: 80.0
```

Discount Solution & Exit Check

- Compare customer explicitly in both alternatives.
- Run the repaired code with adult, student and senior.
- Explain when the indented update runs.
- Before class 2, review slicing, Boolean expressions and branch order.

```
customer = "adult"
price = 100
if (customer == "student"
    or customer == "senior"):
    price *= 0.8
print(price) # 100

# Exit check:
# What is "Python"[1:4]?
# Why test score >= 90 before >= 60?
# Answers: yth; the first match wins.
```